练习：发布 Cars CRUD Docker 镜像（使用 GitHub Actions）

目标：扩展你现有的 .github/workflows/tests.yml，使它仍然测试以 main 为目标的 pull request，并且在推送到 main 且测试通过之后，构建 Cars CRUD Docker 镜像并发布到你的 Docker Hub 账户。

这是一个自己动手写的练习，提供的是要求和提示，而不是一份完整的工作流。

前提条件：你已完成 TESTING_WORKSHOP.md、GITHUB_ACTIONS_GUIDE.md、GITHUB_ACTIONS_EXERCISE.md 和 DOCKER_GUIDE.md。你现有的 tests.yml 已经能针对 pull request 成功运行单元、集成、API 和 E2E 测试。你有一个 Docker Hub 账户和一个 cars-crud 仓库。

1. 完成的工作流应当做什么

事件	预期结果
打开或更新以 main 为目标的 PR	运行全部四个测试组。不发布镜像。
将该 PR 合并到 main（或推送到 main）	运行全部四个测试组。如果通过，则构建并发布镜像。
main 上的测试失败	将测试作业标记为失败，并跳过发布。

为什么不在 pull request 中发布？发布会使用 Docker Hub 访问令牌。请把该凭据排除在执行未合并的 pull request 更改的作业之外。此外，GitHub 通常不会向由 fork 的 pull request 触发的工作流提供仓库 secrets。

2. Docker Hub 与课程的 Azure 拉取代理

在中国本地工作时，从 Docker Hub 拉取可能会很慢。课程为受支持的缓存镜像提供了一个 Azure 中间层，地址为 efreiwut.azurecr.io：

```
# 普通 Docker Hub 拉取
docker pull python:3.12

# 通过课程代理的等价镜像/标签
docker pull efreiwut.azurecr.io/cache/library/python:3.12
```

Dockerfile 同样可以使用任一种基础镜像路径：

```
FROM python:3.12
# 或者，用于通过 Azure 进行本地访问：
```

```
# FROM efreiwut.azurecr.io/cache/library/python:3.12
```

Cars CRUD 应用使用 python:3.12-slim，因此匹配的 Azure 路径以 /python:3.12-slim 结尾。在 Docker 指南中，Dockerfile 接受 BASE_IMAGE 作为构建参数：本地构建可以选择 Azure 代理，而默认仍是公共 Docker Hub 镜像。

对于本练习：

- 本地开发：如果 Azure 路径能改善 Python 基础镜像的拉取，就使用它。
- GitHub Actions：托管运行器通常可以直接从 Docker Hub 拉取 python:3.12-slim，因此无需代理设置。如果 Azure 代理可访问且经过有意配置，工作流也可以使用它。
- 发布：将完成的 Cars CRUD 镜像推送到你的 Docker Hub 仓库 (YOUR_DOCKER_ID/cars-crud:latest)，而不是 efreiwut.azurecr.io。课程代理不是你 Docker Hub 发布凭据的替代品，也不能保证代理个人 Docker Hub 仓库。
- 生产环境：组织通常使用自己的注册表和镜像 URL，例如 registry.example.com/team/cars-crud:v1.

如果你的 Dockerfile 使用了指南中的构建参数，那么在 GitHub Actions 中无需设置 build-args:。代理只是拉取基础镜像的
替代来源。

3. 准备你的练习分支

在你现有的 cars-crud/ 仓库中：

```
git switch main
git pull --ff-only origin main
git switch -c docker-image-workflow
```

确保以下文件存在于该分支中：

```
cars-crud/
├── Dockerfile
├── .dockerignore
├── requirements.txt
├── app/
└── .github/
    └── workflows/
        └── tests.yml
```

如果你还没有提交 Dockerfile 和 .dockerignore，请按照 Docker 指南添加它们。Dockerfile 和应用文件必须已提交，这样 GitHub 才能构建出与你本地构建相同的
镜像。

4. 配置 GitHub 仓库 secrets

1. 在 Docker Hub 中，创建一个具有 Repo Read & Write 权限的个人访问令牌，如果你还没有的话。
2. 在你的 GitHub 仓库中，打开 Settings > Secrets and variables > Actions。
3. 在 Repository secrets 下，选择 New repository secret 并创建：

Secret 名称	Secret 值
`DOCKERHUB_USERNAME`	你的 Docker Hub ID / 用户名。
`DOCKERHUB_TOKEN`	你的 Docker Hub 个人访问令牌。

用户名也可以存储为 GitHub 变量，但为了一致性，本练习使用上述两个名称。
不要将令牌放进 YAML、已提交的 .env 文件或终端截图中。

5. 工作流要求

修改现有文件：.github/workflows/tests.yml。不要创建一个不等待测试就发布镜像的无关工作流。

1. 保留现有的 tests 作业及其四个测试组：
   - `tests/unit/`
   - `tests/integration/`
   - `tests/api/`
   - `tests/e2e/`
2. 保留针对以 main 为目标的 PR 的 pull_request 事件。
3. 还要为 main 添加 push，这样合并 PR 会在更新后的 main 分支上启动一次运行。
4. 在 Ubuntu 运行器上添加一个名为 publish 的独立作业。
5. 让 publish 依赖 tests 作业。它必须仅在成功推送到 main 时运行，绝不在 pull request 上运行。
6. 在 publish 中，检出仓库、准备 Docker Buildx、使用 GitHub secrets 登录 Docker Hub、构建仓库的 Dockerfile，并推送一个标记为以下内容的镜像：

   ```
   YOUR_DOCKER_ID/cars-crud:latest
   ```

7. 保持工作流的 GitHub 令牌权限最小化（对仓库内容的读访问对本示例已足够）。失败的测试必须阻止发布。

6. 提示（不是完整解决方案）

提示 A：配置两个事件

这会替换上一个练习中仅限 PR 的触发器：

```yaml
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
```

对于 pull_request，main 是目标分支。对于 push，main 是接收提交的分支。

提示 B：添加一个依赖作业

在 jobs: 下、与 tests: 相同的缩进位置添加 publish。从这个未完成的大纲开始：

```yaml
  publish:
    needs: tests
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      # TODO：检出代码
      # TODO：设置 Docker Buildx
      # TODO：向 Docker Hub 认证
      # TODO：构建并推送镜像
```

needs: tests 表示此作业会等待 tests 作业成功。if 条件确保该作业不会在 pull request 上使用发布凭据。不要从现有作业中删除 tests。

提示 C：选择动作

相关的可复用动作有：

动作	用途
actions/checkout@v7	在作业中提供此仓库。
docker/setup-buildx-action@v4	准备 Docker 镜像构建器。
docker/login-action@v4	使用存储的凭据登录 Docker Hub。
docker/build-push-action@v7	构建 Dockerfile 镜像并将其标签推送到注册表。

对于 docker/login-action，请研究它的 with: username: 和 with: password: 选项。GitHub secrets 使用以下表达式格式读取：

```
${{ secrets.DOCKERHUB_TOKEN }}
```

对于 docker/build-push-action，请研究这些 with: 选项：

```
context: .
push: true
tags: YOUR_DOCKER_ID/cars-crud:latest
```

用配置好的用户名（GitHub secret 表达式也可以）替换固定的 YOUR_DOCKER_ID。
不要在 Dockerfile 中用 --build-arg 或 ENV 传递注册表密码或令牌。你
无需在 publish 作业中再次安装 Python 或 Playwright；tests 作业已经运行过它们了。

可选加分项：除了 latest，再发布一个包含 Git 提交 SHA 的标签，这样你可以识别
出具体是哪个提交生成了该镜像。

7. 打开 PR 并验证两个事件

保存你的更改，然后提交并推送你的分支：

```
git add Dockerfile .dockerignore .github/workflows/tests.yml
git commit -m "Publish tested Cars CRUD image to Docker Hub"
git push -u origin docker-image-workflow
```

如果 Dockerfile 或 .dockerignore 没有更改，Git 在暂存时会直接忽略它们。

1. 在 GitHub 上，打开一个 base 为 main、compare 为 docker-image-workflow 的 pull request。
2. 打开 Checks 标签页或 Actions 标签页。验证单元、集成、API 和 E2E 测试通过。
3. 验证 publish 在 PR 中被跳过，而不是登录 Docker Hub。
4. 将 PR 合并到 main。
5. 打开由推送到 main 触发的新工作流运行。确认测试作业先完成，然后
   发布作业成功。
6. 打开你的 Docker Hub cars-crud 仓库，确认 latest 镜像标签已上传。

如果启用了分支保护，可以要求在合并之前通过测试检查。

8. 证明发布的镜像可用

在安装了 Docker 的本地终端中（替换 YOUR_DOCKER_ID）：

```
docker pull YOUR_DOCKER_ID/cars-crud:latest
docker run -d --name cars-from-hub -p 127.0.0.1:8000:8000 --mount
type=volume,source=cars-from-hub-data,target=/data YOUR_DOCKER_ID/cars-
crud:latest
```

打开 http://localhost:8000/ 并创建一辆车。如果 8000 端口已被你之前的 cars-app
容器占用，请先停止该容器（或本次使用 127.0.0.1:8001:8000）。

应用和 HTML 的行为应当与运行本地构建的镜像时完全一致。

9. 完成检查清单

- [ ] Dockerfile 能在本地构建完整的 FastAPI + HTML 应用。
- [ ] Docker Hub 仓库和 GitHub secrets 已配置。
- [ ] 工作流会测试以 main 为目标的 PR，但不发布。
- [ ] 工作流也会在推送到 main 时运行测试。
- [ ] 发布作业依赖于测试成功。
- [ ] 发布仅在推送到 main 时运行。
- [ ] Docker Hub 包含 YOUR_DOCKER_ID/cars-crud:latest。
- [ ] 拉取并运行该镜像会显示 Cars CRUD 应用。

10. 故障排查

问题	检查什么
发布作业总是被跳过	你查看的是推送到 main 的运行吗？在 PR 上被跳过是正常的。
发布作业在测试失败后仍运行	检查作业级别的 needs: tests，并移除任何 always() 覆盖。
推送时 unauthorized 或 denied	检查仓库名称、Docker Hub 令牌权限以及两个 GitHub secret 名称。
找不到 Dockerfile	确保 Dockerfile 已提交到仓库根目录，且构建上下文是 .。
合并后没有新运行	检查 on.push.branches、YAML 有效性以及仓库的 Actions 设置。
latest 没有更新	检查传给构建动作的 tags 以及 Docker Hub 仓库页面。
浏览器无法访问下载的镜像	检查 docker ps、docker logs cars-from-hub 和主机端口映射。
Python 基础镜像拉取在本地很慢	按照 Docker 指南，使用 BASE_IMAGE 构建参数通过 Azure 代理构建；GitHub 运行器可以直接使用 Docker Hub。
找不到 Azure 代理镜像	检查确切的 /cache/library/python 路径、标签、注册表可达性和访问设置。

参考

- Docker：GitHub Actions 镜像构建与推送
- Docker login action
- Docker build and push action
- GitHub：工作流语法
- GitHub：在 Actions 中使用 secrets